Linux Process Tree Visualizer

Live process hierarchy using the Linux /proc filesystem

Hackathon Project Report

Operating Systems and Systems Programming (25CS2104E)
Faculty: Dr. Muniraju Naidu

Team Members:

Teja Hasini Reddy - 2520030048

Hansika Reddy - 2520030237

Simran Jangid - 2520030166

Anthony Nissy - 2520030064

KLH University (Deemed to be University)

Abstract

Every program running on a Linux system is a process, and every process except the first one is created by another process. Together these parent-child relationships form a tree. Ordinary commands such as ps and top present processes as a flat table, which hides this structure. This report describes the Linux Process Tree Visualizer, a terminal tool written in C that reads the /proc virtual filesystem, rebuilds the hierarchy of running processes and displays it as a color-coded tree that refreshes in real time.

The tool shows the name, process ID (PID), parent process ID (PPID) and state of each process, highlights running, stopped and zombie processes, and prints a summary of their counts. It supports options to include kernel threads, to print a single snapshot, and to show usage help. The program uses only the C standard library and POSIX system calls, needs no configuration, and exits cleanly on Ctrl+C.

1. Introduction

An operating system is responsible for sharing the processor, memory and other resources between many programs at once. Each running program is represented by a process, which has a unique identifier (PID), its own address space and a state that describes what it is currently doing. Understanding how processes relate to one another is a central topic in operating systems, because the relationships explain how programs are started, how resources are inherited and what happens when a program ends.

In Linux, a new process is created when an existing process calls fork(). The new process, called the child, receives its own PID and remembers the PID of its creator as its parent process ID (PPID). Because every process has exactly one parent, the whole system forms a tree. The root of this tree is PID 1, started by the kernel at boot and usually run by init or systemd. When a parent process ends before its children, the children are adopted by PID 1, so the tree always stays connected.

The kernel makes information about every running process available through /proc, a virtual filesystem that exists only in memory. Each running process has a folder named after its PID, and files inside that folder, such as stat, contain the process name, state and PPID as plain text. Reading these files requires no special library or privileges, which makes /proc an ideal data source for a small monitoring tool.

The aim of this project is to turn that raw data into a clear picture. The Linux Process Tree Visualizer scans /proc, links every process to its parent and prints the result as an indented tree. The scan is repeated every few seconds, so the display follows the system as processes start, change state and exit. The project was developed for the hackathon task: create a tool that displays the hierarchy of running processes and their parent-child relationships in real time.

2. Problem Statement and Objectives

Tools such as ps and top list processes in a flat table. A reader has to compare PID and PPID columns by hand to work out which process started which, and the output is only a snapshot that must be rerun to see changes. Problem processes, such as zombies that have finished but were never collected by their parent, are easy to overlook in a long list. The task is therefore to build a tool that shows the hierarchy of running processes and their parent-child relationships in real time.

2.1 Objectives

Objective	How it is achieved
Read process data	Open /proc and read /proc/<pid>/stat directly, with no external tools
Build the hierarchy	Link every process to its parent using PID and PPID
Display clearly	Print a tree showing name, PID, PPID and state on every line
Update in real time	Rescan and redraw every 3 seconds; exit cleanly on Ctrl+C or SIGTERM
Be robust	Skip processes that exit during a scan; handle unusual names and orphans
Be easy to use	One source file, one build command and the options --all, --once and --help

2.2 Comparison with ps and top

Aspect	ps / top	Process Tree Visualizer
Layout	Flat list	Indented tree with branch symbols
Parent link	Needs extra options or columns	PPID shown on every line
Updates	Run again manually	Refreshes automatically every 3 seconds
Problem states	Plain state letters	Colored: running, stopped, zombie
Setup	Installed system utilities	One C file built with gcc

2.3 Scope

The tool is read-only. It observes processes and never changes, signals or kills them. It runs on any Linux system where /proc is mounted and in any terminal that supports ANSI color codes. Kernel threads, which are created by the kernel itself and clutter the tree, are hidden by default and can be shown with the --all option.

2.4 Requirements

Functionally, the tool must display the hierarchy with the name, PID, PPID and state of every process, refresh automatically in real time, and highlight running, stopped and zombie processes. Non-functionally, it must be lightweight (one C file, no external libraries, no configuration) and robust (tolerate processes that exit during a scan and exit cleanly on Ctrl+C).

3. Background Concepts

3.1 Processes, fork and the process tree

A process is created by fork(), which duplicates the calling process. The copy gets a new PID and records the caller as its parent. Usually the child then calls exec() to load a different program. For example, when a user types a command in a shell, the shell forks a child and the child executes the command, so the command appears as a child of the shell in the tree. PID 1 is the ancestor of all user-space processes, and kernel threads are grouped under kthreadd (PID 2).

3.2 The /proc/<pid>/stat file

The stat file contains a single line of space-separated fields. The visualizer uses only the first four. Because the name field can itself contain spaces and brackets, the parser must find the first opening bracket and the last closing bracket before reading the remaining fields.

Field	Content	Used for
1	PID	Identifies the process
2	Name inside ( )	Shown in the tree
3	State letter	Color coding and summary counts
4	PPID (parent PID)	Builds the parent-child link

3.3 Process states

State	Meaning	Terminal color
R	Running or ready to run	Green
S	Sleeping, waiting for an event	Default
D	Uninterruptible sleep, usually waiting for I/O	Default
T	Stopped by a signal or a debugger	Yellow
Z	Zombie: finished, but the parent has not collected its exit status	Red

A process normally switches between running and sleeping many times a second. The states worth watching are D, T and Z, because they point to blocked input and output, paused jobs and parents that have not collected finished children.

3.4 Zombie, orphan and stopped processes

A zombie process has finished, but its parent has not yet called wait() to collect its exit status, so its entry stays in the process table; a growing number of zombies usually signals a bug in the parent. An orphan is a process whose parent has ended; the kernel reassigns it to PID 1. A stopped process has been paused by a signal such as SIGSTOP or by a debugger. The visualizer colors zombies red and stopped processes yellow so these cases are easy to notice.

3.5 Kernel threads

Kernel threads are created by the kernel for internal work and are children of kthreadd (PID 2). Because they are not user programs, they are hidden by default and shown with --all.

4. System Design

The visualizer repeats one simple cycle: scan the processes, parse their data, sort them, print the tree and wait. Each step is handled by its own function, which keeps the program short and easy to explain.

4.1 Processing steps

Step	What happens	Function
1 Scan	List every folder in /proc whose name is a number	`scan_processes()`
2 Parse	Read name, state and PPID from the stat file	`read_process()`
3 Sort	Order the array of processes by PID	`qsort, compare_processes()`
4 Find roots	Select processes whose parent is not in the list	`find_process()`
5 Print tree	Print children recursively with branch symbols	`print_node()`
6 Repeat	Clear the screen, wait 3 seconds and rescan	`main loop, nanosleep`

4.2 Program flow

1. Start: read the command-line options (--all, --once, --help) and install handlers for SIGINT and SIGTERM.
2. scan_processes(): open /proc, read the stat file of every numeric folder and skip kernel threads unless --all is used.
3. Sort the stored processes by PID so that siblings always appear in the same order.
4. display_tree(): count running, stopped and zombie processes and print the header with the legend.
5. print_node(): print each root process, then recursively print its children with branch characters.
6. Sleep for 3 seconds and repeat from step 2 until Ctrl+C or SIGTERM is received.

4.3 Data structure

Each process is stored as a record in a fixed-size array of up to 20,000 entries. The printed flag marks a process that has already been drawn so that it is never repeated. A fixed array avoids dynamic memory, so the program cannot leak memory. Sorting by PID keeps the order of siblings stable, and a process whose parent is missing is treated as a root so that orphans stay visible.

```
typedef struct {
    int  pid;
    int  ppid;
    char name[NAME_SIZE];
    char state;
    bool printed;
} Process;
```

5. Implementation

The program is written in C and built with: gcc -Wall -Wextra -O2 -o ptree ptree.c. It includes only standard headers and POSIX interfaces (dirent.h, signal.h, time.h, unistd.h). The most important parts of the code are explained below.

5.1 Reading a process

read_process() opens /proc/<pid>/stat and reads the line into a buffer. The name is taken from between the first opening bracket and the last closing bracket, so names that contain brackets are handled correctly. The state letter and PPID are then read from the text after the closing bracket.

```
char *left  = strchr(buffer, '(');
char *right = strrchr(buffer, ')');

memcpy(p->name, left + 1, length);
p->name[length] = '\0';

sscanf(right + 1, " %c %d", &p->state, &p->ppid);
```

5.2 Printing the tree

print_node() prints one process, then calls itself for each unprinted child. A prefix string grows by four characters at every level, adding spaces below the last child and a vertical bar below the others, so deeper branches line up under their parent. The count of remaining children decides whether a child receives the middle branch symbol or the end-of-branch symbol. A depth limit of 1000 and the printed flag guarantee that the recursion always ends.

```
if (is_root) {
    print_process(&processes[index]);
} else {
    printf("%s%s ", prefix,
           is_last ? "\u2514\u2500\u2500" : "\u251C\u2500\u2500");
    print_process(&processes[index]);
}
```

5.3 Functions in the program

Function	Purpose
read_process()	Parse /proc/<pid>/stat into name, state, PID and PPID
scan_processes()	Walk /proc, filter kernel threads and sort the list
compare_processes()	Comparison used by qsort to order by PID
find_process()	Check whether a parent PID exists in the list
count_children()	Count unprinted children to choose the branch symbol
print_node()	Recursively print a process and its children
display_tree()	Print the header, statistics and the full tree
handle_signal()	Stop the main loop on Ctrl+C or kill

6. Features, Usage and Output

6.1 Features

The tree uses branch characters to show depth and siblings, and every line shows the process name, PID, PPID and state. Running processes are shown in green, stopped processes in yellow and zombie processes in red. A header summarises the refresh interval, the number of processes found and how many are running, stopped or zombie. The program refreshes automatically every 3 seconds, and Ctrl+C prints a stop message and leaves the terminal in a normal state.

6.2 Command-line usage

Command	Effect
`gcc -Wall -Wextra -O2 -o ptree ptree.c`	Compile the program
`./ptree`	Live view, refreshed every 3 seconds, kernel threads hidden
`./ptree --all`	Include kernel threads in the tree
`./ptree --once`	Print one snapshot and exit
`./ptree --help`	Show usage; Ctrl+C stops the live view

6.3 Output format

Each line has the form: name (PID, PPID, State). Children are indented below their parent. The listing below is an illustration of the format, not a captured run.

```
systemd (PID: 1, PPID: 0, State: S)
|-- sshd (PID: 812, PPID: 1, State: S)
|   `-- bash (PID: 950, PPID: 812, State: S)
`-- cron (PID: 640, PPID: 1, State: S)
```

Illustrative output (the program draws the branches with box-drawing characters)

The header above the tree reports the refresh interval or snapshot mode, whether kernel threads are shown, the number of processes found, and the counts of running, stopped and zombie processes, followed by a color legend. In live mode the screen is cleared before each redraw and the line Press Ctrl+C to exit is printed below the tree.

6.4 Header summary

Header line	Meaning
Mode / Kernel threads	Refresh interval or single snapshot; kernel threads shown (--all) or hidden
Processes found	Number of processes in the tree
Running / Stopped / Zombies	Counts of processes in states R, T and Z
Legend	Color key: running green, stopped yellow, zombie red

7. Challenges and Solutions

Challenge	Solution
Names with spaces or brackets	Use the first '(' and the last ')' of the stat line
A process exits during a scan	Skip any file that cannot be opened or read
Orphaned processes	Treat a process with a missing parent as a root
Very large systems	Fixed limit of 20,000 processes avoids unbounded memory use
Inconsistent data	Printed flag and depth limit make recursion always end
Kernel thread clutter	Hidden by default; shown only with --all
Interrupted sleep	nanosleep resumes with the remaining time unless stopping

8. Verification Scenarios

The behaviour of the tool can be checked on any Linux machine with the following scenarios. Each is expected to produce the result shown.

Scenario	Expected result
Run ./ptree --once	One tree rooted at PID 1 and a header with the process counts
Start sleep 100 & then run the tool	sleep appears as a child of the shell that started it
Stop a job with kill -STOP <pid>	The process state changes to T and is shown in yellow
Create a zombie with a short test program	The zombie shows state Z in red under its parent
Run ./ptree --all	Kernel threads appear under kthreadd (PID 2)
Press Ctrl+C in live mode	The stop message is printed and the program exits normally

9. Applications, Limitations and Future Scope

Applications. The tool helps students see how fork() builds parent-child relationships and how init adopts orphans. Administrators can see which service or shell owns a group of processes, find zombies and the parent that failed to collect them, and save a snapshot with --once for reports.

Limitations. The tool shows process identity and state only; it does not show CPU or memory use, and it offers no interactive control. It is limited to 20,000 processes and to Linux systems that provide /proc.

Future scope. Possible extensions are filtering by user, showing CPU and memory per process, searching by name, collapsing branches, and an interactive option to send signals.

10. Conclusion

The Linux Process Tree Visualizer meets the hackathon task. It reads the /proc filesystem, rebuilds the parent-child hierarchy of all running processes and shows it as a color-coded tree that refreshes every 3 seconds. Zombie and stopped processes stand out immediately, and the summary header gives a quick overview of the system.

During the project the team applied several operating system and systems programming concepts: process creation and the process tree, process states, the /proc filesystem, directory traversal, safe text parsing, recursion and signal handling. The program is lightweight, uses no external libraries and stays stable on a busy system because unreadable files and out-of-range values are skipped.

The result is a small but practical tool that makes an abstract concept visible, and it can be extended with resource usage and interactive features in future work.

References

1. Linux manual pages: proc(5), the /proc filesystem and the stat file format.

2. Linux manual pages: fork(2), wait(2), signal(7) and sigaction(2).

3. Linux manual pages: ps(1) and pstree(1), existing tools with similar goals.

4. POSIX documentation: opendir, readdir and nanosleep.

5. Course material: Operating Systems and Systems Programming (25CS2104E), KLH University.

Team Details

Name	Roll number
Teja Hasini Reddy	2520030048
Hansika Reddy	2520030237
Simran Jangid	2520030166
Anthony Nissy	2520030064

Acknowledgement

The team thanks Dr. Muniraju Naidu for guidance throughout the course Operating Systems and Systems Programming (25CS2104E), and KLH University for the opportunity to take part in the hackathon.

Faculty: Dr. Muniraju Naidu | Course: Operating Systems and Systems Programming (25CS2104E)